p3497R0 - guarded objects

guarded objects - make the relationship between objects and their locking mechanism explicitly expressable and hard to use incorrect

11 November 2024

Authors:

- Jan Wilmans (janwilmans at gmail.com)

Audience:

- Library Evolution Working Group (Design & Target)
- Library Working Group (Wording and Consistency)

Project:

- ISO JTC1/SC22/WG21: Programming Language C++

Current version:

- https://github.com/janwilmans/proposals/blob/master/pXXXXR0_guarded_objects.md

Reply To:

- Jan Wilmans janwilmans@gmail.com

Abstract

In multithreaded programming locking mechanisms are used to prevent concurrent access to data. Common practice is to create a locking mechansim, lets say an **std::mutex** along side the **data** it is protecting. However, the relationship between the mutex and the data is only implied and expressed in code only by naming variables and/or 'doing it right' in all places. This proposal improves this by providing a way to clearly express the relationship and make it impossible to access the data without locking its associated guarding mechanism.

Revision History

Revision 0

Initial version

Motivation

Encapsulation of locking logic

The `guarded` type encapsulates the locking mechanism and its data, ensuring that lock is acquired and released properly when accessing the protected data. By requiring access to the data through the guarded type, you make it harder (or impossible) to accidentally access the data without properly locking it first.

Simplified API

By combining the the data and its locking mechanism in one type they have the same lifetime and the API for interacting with the protected data becomes clearer. Users don't have to worry about manually locking and unlocking. Instead, the locking and unlocking can be handled internally within the type. This simplifies the API and reduces the cognitive load.

Strong ownership semantics / RAII guarantees

The only wat to access data is to call one of the locking functions, these functions either return an object (a unique_ptr-like object) that owns the lock, or allows you to pass in an operation that is exectured while holding the lock. It can be used in a familiar way, just like you would use a smart pointer, or when passing in the operation, not dealing with the lock directly at all.

The locking mechanism is based on RAII (Resource Acquisition Is Initialization), the type handles acquiring and releasing the lock in a scoped manner. This makes sure the lock is always released no matter how you leave the scope, returning of by an exception for example.

Separation of concerns

The guarded type eliminates the need to embed thread-safety directly into the design of classes. Such implementations are pessimizing single-threaded use and have locking overhead on every API call. An example of a 'synchronized' queue:

```
#include <queue>
#include <mutex>

#include <cs_plain_guarded.h>

template <typename T>
class SynchronizedQueue
{
public:
    bool Empty() const
    {
        std::unique_lock<std::mutex> lock(m_mtx);
        return m_q.empty();
    }

    void Push(T t)
    {
        std::unique_lock<std::mutex> lock(m_mtx);
        m_q.push(std::move(t));
    }

    T Pop()
    {
        std::unique_lock<std::mutex> lock(m_mtx);
        T t(m_q.front());
        m_q.pop();
        return t;
    }

    mutable std::mutex m_mtx;

    // here there is the implied agreement, that 'm_mtx' should be locked
    // before accessing m_q, however nothing enforces this.
    std::queue<T> m_q;
};
```

Notice that the code above is fragile in the sense that every public method **must** acquire the lock and forgetting to do so is only checkable by reviewing.

Alternative with a guarded lock from the [cs_libguarded] library:

```
template <typename T>
using GuardedQueue = libguarded::plain_guarded<std::queue<T>>;

bool example()
{
    GuardedQueue<int> guarded_queue;

    // it is not possible to access the std::queue directly, without calling lock()
    return guarded_queue.lock()->empty();
}

void example2()
{
    GuardedQueue<int> guarded_queue;

    // it is not possible to access the std::queue directly, without calling lock()
    auto locked_q = guarded_queue.lock();

    // execute code with the lock held.
    if (locked_q->empty())
    {
        // do things
    }

    // lock leaves scope.
}
```

[compiler explorer link]

For demonstration purposes, here is a naive implementation of a 'guarding' class that allows you to pass in a function:

```
// Naive example how an operation could be passed in, to perform a set of operations while holding the lock

#include <mutex>
#include <string>
#include <iostream>

template<typename T>
class naive_guarded {
private:
    T data;
    mutable std::mutex mtx;

public:
```

```
    template<typename Func>
    auto with_lock(Func&& func) {
        std::lock_guard<std::mutex> lock(mtx);
        return func(data);  // Pass the data to the provided function.
    }
};

int main() {
    naive_guarded<std::string> naive_guarded_string;

    // Modify the string safely.
    naive_guarded_string.with_lock([](std::string& str) {     // locking overhead
        str = "Hello, World!";
    });

    // Read the string safely.
    naive_guarded_string.with_lock([](const std::string& str) { // locking overhead
        std::cout << str << '\n';
    });

    return 0;
}
```

The example above is a demostration only

compiler explorer link

Easier to reason about the code / better readability

When locking is tightly coupled with the data it protects, it becomes easier to reason about the behavior of the code. A guarded makes it clear that access to the std::string is controlled and synchronized. There is also no question about what the lock is protecting, this makes the code more understandble and maintainable. The guarded type abstracts away the details of how the concurrency mechanisms are implemented. Users of the type do not need to worry about whether the data is protected by a mutex, spinlock, or some other concurrency primitive; they get a guarantee the data access is synchronized properly.

Design considerations

Overview

The goal of this proposal is to provide a type:

1) better readability and maintainability: by explicitly expressing the relationship between data and its guarding mechanism 2) better thread safely / hard to use incorrectly: by making it impossible to access the data without locking its guarding mechanism 3) make unlocking automatic and exception safe: by returning a RAII object that has ownership of the locked state

Synopsis

An example of what the result could look like:

```
#include <string>
#include <iostream>

#include "https://raw.githubusercontent.com/copperspice/cs_libguarded/master/src/cs_plain_guarded.h"

int main() {
    libguarded::plain_guarded<std::string> guarded_string;

    auto accessor = guarded_string.lock();  // as long as 'accessor' remains in scope, the mechanism remains locked.

    // Modify the string safely.
    *accessor = "Hello, World!";

    // Read the string safely.
    std::cout << *accessor << '\n';

    return 0;
}  // accessor leaves scope, automatically unlocking
```

compiler explorer link

Demonstration of clear separation of the class implementation and the sychronisation of the class.

```
#include <string>
#include <vector>
#include <iostream>

#include "https://raw.githubusercontent.com/copperspice/cs_libguarded/master/src/cs_plain_guarded.h"

struct person
```

```
{
    std::string name;
};

using contacts_t = std::vector<person>;

void print(const contacts_t& contacts)
{
    for (const auto& person: contacts)
    {
        std::cout << person.name << '\n';
    }
}

int main()
{
    libguarded::plain_guarded<contacts_t> synchronized_contacts;  // specifiy only synchronized access to contacts_t is possible
    print(*synchronized_contacts.lock());                         // use of the lock
    return 0;
}
```

Discuss customizable [thread.req.lockable] interface with Cpp17BasicLockable requirements

```
std::guared<T>;
std::guared<T, L>;   // The type of second argument satisfies the [thread.req.lockable] interface
```

Bikeshedding

This section is for naming, conventions and pinning down details to make it suitable for the standard.

As a suggestion: `std::guarded<T>` would express the intent.

Acknowledgements

- this proposal is based on the the work of Ansel Sermersheim and his https://github.com/copperspice/cs_libguarded library

References

- https://github.com/copperspice/cs_libguarded
- [thread.req.lockable] Cpp17BasicLockable requirements